

# Resumen de clase

## Instanciación vs. uso



2° cuatrimestre 2008

## Indice

|                                                                                 |          |
|---------------------------------------------------------------------------------|----------|
| <b>ENUNCIADO .....</b>                                                          | <b>3</b> |
| <b>RESOLUCIÓN TRADICIONAL .....</b>                                             | <b>4</b> |
| <b>RESOLUCIÓN EN OBJETOS .....</b>                                              | <b>5</b> |
| 1) TENGO UN IF PARA CADA STRATEGY .....                                         | 5        |
| 2) USO UN MAPA .....                                                            | 6        |
| 3) PARAMETRIZO LA CLASE PARA INSTANCIAR EL OBJETO (REFLECTION) .....            | 8        |
| <i>new CriterioConservador()</i> vs. <i>CriterioConservador new de St</i> ..... | 9        |
| ¿QUÉ PASA SI LOS STRATEGIES TIENEN ESTADO DIFERENTES ENTRE SÍ? .....            | 10       |

## Enunciado

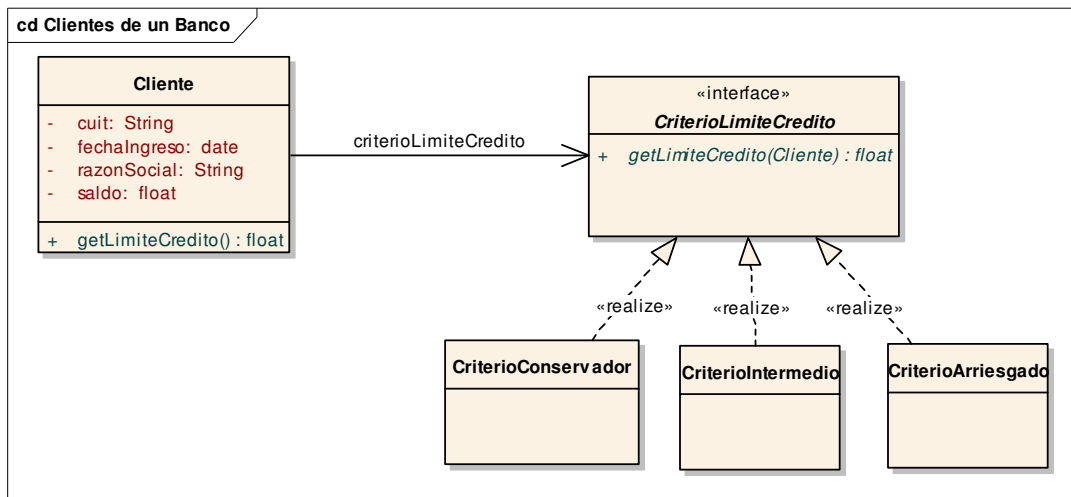
Comenzamos con un ejemplo sencillo. Un cliente de un banco que tiene

- Razón Social
- CUIT
- Fecha de ingreso
- Saldo
- y el criterio para calcular el monto máximo si pide un crédito.

Hay 3 tipos de criterios para establecer el límite de crédito:

- conservador
- arriesgado
- intermedio.

Los cálculos de cada uno se los dejamos librados a vuestra imaginación, no interesan a los fines didácticos de la clase.



- Volvemos sobre el Strategy, está bueno tener un objeto que en runtime me devuelva el límite de crédito y lo pueda cambiar.
- Armamos un escenario posible: o sea, objetos que están en mi ambiente. Los represento ¿dónde?
  - en un Workspace (en St) / Scrapbook Page (en Java, aunque nadie lo hace)
  - en un main (en Java)
  - en un TestCase (con JUnit / SUnit)

o bien, hacemos una pantalla que genera un cliente.

Metemos en la interfaz de usuario cada atributo: razón social, cuit, etc.

Ahora, ¿qué control usamos para setear el Strategy?



Tiramos ideas: radio buttons , combo boxes  , etc.

La pantalla se va a parecer a algo como esto:

Actualización de Clientes

Razón Social: RABUFETTI Y ASOCIADOS

CUIT: 30-1485975-1

Fecha Ingreso: 10/10/2008

Criterio Límite de Crédito: CONSERVADOR

Aceptar Cancelar

ALTA CAPS

Hay entonces dos momentos en la aplicación:

- en el `getLimiteCredito()` de `Cliente` yo **uso** los distintos criterios que son objetos polimórficos
- en un momento previo alguien definió qué estrategia de límite de crédito le correspondía a ese cliente.

Cuando les presentamos el Strategy pusimos foco en el uso de los objetos. Ahora vamos a poner foco en cómo armamos el escenario para que mis objetos sean polimórficos.

Lo copado es que pueda delegar la creación de esos objetos tanto como pueda para que siga siendo feliz mi aplicación (y por ende, yo también).

## Resolución tradicional

En la programación tradicional (no objetosa), el combo tiene asociado un valor fijo que el programador hardcodea en los futuros ifs.

**Por ejemplo:** el combo muestra como opciones: "Conservador", "Intermedio" o "Arriesgado" pero guarda internamente los números 1, 2 y 3 (o "C", "I" y "A"). Después cuando se resuelve el límite de crédito hay un súper switch:

```
switch (tipoLimiteCredito) {  
  case 1: // Conservador  
    return --- blahh ---;  
  case 2: // Intermedio  
    return --- blehh ---;  
  case 3: // Arriesgado  
    return --- blihh ---;  
  default: // Ouch!  
}
```

**Nota:** acá se ve que cuando actualizo el registro en la tabla, guardo el 1, el 3, o el valor que corresponda y no tengo mucha lógica más que cumplir. La lógica se posterga hasta el uso de ese valor en el cálculo del límite de crédito. Veamos qué pasa en objetos...

## Resolución en Objetos

Ok, en objetos, usamos el Strategy. Entonces la pregunta es: ¿cómo transformo el combo en el Strategy?

Encontramos varias opciones:

### 1) Tengo un *if* para cada Strategy

#### 1.a) Hay un switch donde creo a mano cada Strategy.

El combo tiene pares clave-valor.

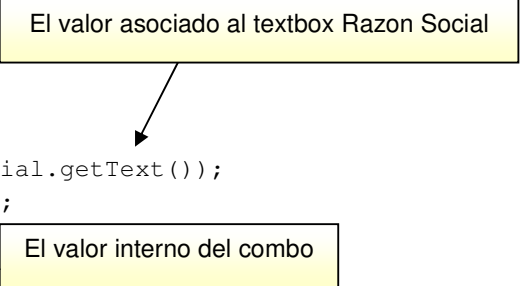
En la clave: valor interno asociado "A", "C" o "I".

En el valor que muestro por pantalla: "Arriesgado", "Conservador" o "Intermedio".

#### Código de la pantalla <sup>1</sup>

```
public void aceptar() {
    Cliente cliente = new Cliente();
    cliente.setRazonSocial(txtRazonSocial.getText());
    cliente.setCUIT(txtCUIT.getText());
    cliente.setFechaIngreso(...);

    if (combo.getSelectedOption().equals("C")) {
        cliente.setCriterioLimiteCredito(new CriterioConservador());
    }
    if (combo.getSelectedOption().equals("A")) {
        cliente.setCriterioLimiteCredito(new CriterioArriesgado());
    }
    if (combo.getSelectedOption().equals("I")) {
        cliente.setCriterioLimiteCredito(new CriterioIntermedio());
    }
}
```



#### 1.b) Tengo varios métodos en la clase Cliente para instanciar el Strategy.

#### Código de la pantalla

```
public void aceptar() {
    Cliente cliente = new Cliente();
    ...
    if (combo.getSelectedOption().equals("C")) {
        cliente.setCriterioLimiteCreditoConservador();
    }
}
```

---

<sup>1</sup> Independientemente de la tecnología que se elija para desarrollar la interfaz de usuario, la pantalla tendrá asociado un código muy similar al que vemos a continuación.

```
if (combo.getSelectedOption().equals("A")) {  
    cliente.setCriterioLimiteCreditoArriesgado();  
}  
if (combo.getSelectedOption().equals("I")) {  
    cliente.setCriterioLimiteCreditoIntermedio();  
}
```

#### #Cliente

```
public void setCriterioLimiteCreditoConservador() {  
    criterioLimiteCredito = new CriterioConservador();  
}  
  
... etc ...
```

**Ventaja:** disminuyo el acoplamiento: el encargado de conocer a sus criterios es el cliente. La pantalla no necesita saber las clases concretas de cada Strategy.

Por otra parte es tentador hablar un poco que si estoy haciendo el ABM de Facturas, no tengo que horrorizarme si hay acoplamiento entre esa pantalla y la clase Factura.

De todas maneras sigo teniendo el if y la interfaz para crear los strategies deja de ser polimórfica, si agrego un strategy nuevo... tengo que crear un setCriterioLimiteCreditoXXXXX en Cliente, con lo cual deja de ser "transparente".

Igual decimos: en la creación los criterios no tienen que ser necesariamente polimórficos (sobre todo si el criterio arriesgado tiene información distinta al criterio conservador).

Lo mismo pasa si:

1.c) ubicamos los métodos setCriterioLimiteCreditoXXXXX en la clase CriterioLimiteCredito (métodos de clase/estáticos: uno por cada subclase). La ventaja de esto es que el cliente deja de conocer las instancias concretas de los strategies, pero además en cada combo de la aplicación no tengo que repetir el mismo código

→ **ONCE AND ONLY ONCE** (Sólo tengo que acordarme cada vez que genero un nuevo Strategy).

## 2) Uso un mapa

El mapa mantiene la relación opción del combo → objeto a crear.

| Clave | Valor                      |
|-------|----------------------------|
| 1     | new CriterioConservador(); |
| 2     | new CriterioArriesgado();  |
| 3     | new CriterioIntermedio();  |

#### Código de la pantalla

```
public void aceptar() {  
    Cliente cliente = new Cliente();  
    ...  
    CriterioLimiteCredito criterioLimite = (CriterioLimiteCredito)  
    mapaCriteriosLimiteDeCredito.get(combo.getSelectedOption());  
    cliente.setCriterioLimiteCredito(criterioLimite);  
}
```

¿Cómo cargo el mapa?

2.a) A mano, programándolo en el initialize (o similar) de la pantalla.

```
public void initialize() {  
    Map mapaCriteriosLimiteDeCredito = new HashMap();  
    mapaCriteriosLimiteDeCredito.put('C', new CriterioConservador());  
    mapaCriteriosLimiteDeCredito.put('A', new CriterioArriesgado());  
    mapaCriteriosLimiteDeCredito.put('I', new CriterioIntermedio());  
}
```

Esto implica que en cada pantalla donde haya un combo de criterios hay que copiar ese código (no respeta el Once and Only Once).

2.b *Primer refactor*) Pedimos a otro objeto (digamos ListProvider, o algo así) que nos devuelva el mapa (sin necesidad de guardarlo en la pantalla). En el código de la pantalla nos queda:

```
String clave = combo.getSelectedOption();  
CriterioLimiteCredito limite = (CriterioLimiteCredito)  
    ListProvider.getLimitesDeCredito().get(clave);  
cliente.setCriterioLimiteCredito(limite);
```

y en la clase ListProvider (o bien en la clase CriterioLimiteCredito):

```
public static Map getLimitesDeCredito() {  
    Map mapaCriteriosLimiteDeCredito = new HashMap();  
    mapaCriteriosLimiteDeCredito.put('C', new CriterioConservador());  
    mapaCriteriosLimiteDeCredito.put('A', new CriterioArriesgado());  
    mapaCriteriosLimiteDeCredito.put('I', new CriterioIntermedio());  
    return mapaCriteriosLimiteDeCredito;  
}
```

Este pequeño refactor permite compartir el mapa entre varias pantallas.

¿Pero qué pasa con los valores que se muestran en cada ítem del combo?

Hay que agregarlos a mano en la lista de valores del combo:

"C" es "Conservador",

"A" es Arriesgado,

"I" es Intermedio.

Uuuh! Eso es feo.

"CONSERVADOR" = "C" = new CriterioConservador()

y estamos poniéndolo en dos lugares distintos.

Dos opciones:

2.b.1) Que cada Strategy me sepa decir qué descripción mostrarle al usuario.

**#CriterioConservador**

```
public String getDescripcion() {  
    return "CONSERVADOR";  
}
```

2.b.2) Cambiar el mapa para que tenga la descripción:

```
public static Map getLimitesDeCredito() {  
    Map criterios = new HashMap();  
    criterios.put("Conservador", new CriterioConservador());  
    criterios.put("Arriesgado", new CriterioArriesgado());  
    criterios.put("Intermedio", new CriterioIntermedio());  
    return mapaCriteriosLimiteDeCredito;  
}
```

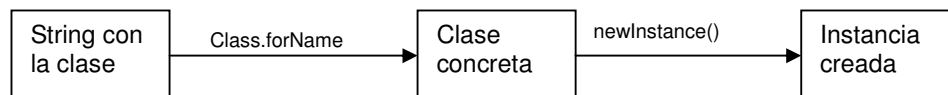
Con 2.b.1 y 2.b.2) usás el mapaCriteriosLimiteDeCredito tanto para llenar el combo como para saber qué objeto Criterio instanciar.

### 3) Parametrizo la clase para instanciar el objeto (Reflection)

#### Código de la pantalla

El combo tiene:

- en el valor la descripción "CONSERVADOR", "ARRIESGADO", etc. que se muestran por pantalla
- como id los nombres de cada clase a crear (un String: "ar.edu.utn.frba.tadp.ej8.CriterioConservador"), que lo transformo a un objeto Clase (CriterioConservador.class) que entiende el mensaje newInstance() para crear un objeto.



```
Cliente cliente = new Cliente();  
...  
String clave = combo.getSelectedOption();  
Class strategyClass = Class.forName(clave);  
CriterioLimiteCredito criterioLimite = (CriterioLimiteCredito)  
    strategyClass.newInstance();  
cliente.setCriterioLimiteCredito(criterioLimite);
```

¿Cómo cargo las opciones del combo?

A mano.

O puedo usar 2.b) o bien

puedo configurarlo en un archivo (puede ser con formato xml) y que eso quede afuera de la aplicación.

```
<xml>  
  <criterios>  
    <criterio descripcion="Conservador"  
      class="ar.edu.utn.frba.tadp.ej8.CriterioConservador">  
    <criterio descripcion="Arriesgado"  
      class="ar.edu.utn.frba.tadp.ej8.CriterioArriesgado">  
    <criterio descripcion="Intermedio"  
      class="ar.edu.utn.frba.tadp.ej8.CriterioIntermedio">  
  </criterios>  
</xml>
```

**Importante:** recordar que el nombre completo de una clase = nombre del package + nombre "corto" de la clase. El nombre completo de la clase es el que permite al Class Loader encontrar una clase e instanciarle un objeto.

### **new CriterioConservador() vs. CriterioConservador new de St**

palabra reservada/no es un mensaje vs. método de clase/puede ser redefinido/puede ser polimórfico en un contexto.

Revisamos el concepto de casteo: newInstance() devuelve un Object.

Preguntamos: ¿hay polimorfismo?

Claro, todos los objetos que tienen constructores sin parámetros son polimórficos para este caso. Pero al setear un criterio de límite de crédito no puedo pasarle un Object. Yo sé que es un CriterioLimiteCredito, entonces... lo casteo.

Claro, si en el xml alguien me pone class="ar.edu.utn.frba.tadp.ej8.Banana" esto se rompe.

**Moraleja:** hay que saber cuándo usar cada herramienta.

**Importante 2:** newInstance() requiere que los strategies tengan constructores públicos sin parámetros (es parte del contrato que hay que respetar).

#### **Ventaja:**

- desaparecieron los ifs, si agrego un strategy más lo único que tengo que hacer es agregar una opción más en el combo/ListProvider/xml que apunte a la clase correspondiente.

#### **Desventajas:**

- hay mayor indirección (no queda tan claro en el código qué tipo de strategy se está asignando)
- al pasar la clase a instanciar como un String no estamos chequeando contra un compilador que la clase sea la correcta (puede estar bien el nombre pero no el package, etc.etc.) Es más fácil refactorizar cuando lo tenés en código

### ***¿Qué pasa si los strategies tienen estado diferentes entre sí?***

Ej: en el criterio arriesgado tenés un porcentaje de riesgo, distinto para cada cliente. El primer corolario es que no son polimórficos para crearse, pero sí para usarse (nuevamente aparecen los dos momentos). Entonces, las pantallas no son polimórficas, y pasa algo como:

The screenshot shows a web form titled "Actualización de Clientes". It contains several input fields: "Razón Social" with the value "RABUFETTI Y ASOCIADOS", "CUIT" with "30-1485975-1", and "Fecha Ingreso" with "10/10/2008". Below these is a section titled "Datos límite de crédito" which contains a dropdown menu labeled "Criterio Límite de Crédito" with "CONSERVADOR" selected. At the bottom of the form are "Aceptar" and "Cancelar" buttons. A status bar at the very bottom shows "ALTA CAPS".

This screenshot shows the same "Actualización de Clientes" form, but with the "Criterio Límite de Crédito" dropdown set to "ARRIESGADO". This selection has dynamically revealed additional fields: a slider control for "% de riesgo" and a text input field containing the value "20". The "Aceptar" and "Cancelar" buttons remain at the bottom, and the "ALTA CAPS" status bar is still present.

(cuando cambiás la selección del combo aparecen distintos campos)  
Y ya no es tan inocente la instanciación del Strategy y hay que evaluar bien si se justifica hacer todo dinámicamente.